NATIONAL TEACHERS COLLEGE

PC16 – Mobile Development with Lab

Week 10: Object-Oriented Programming in Dart (Part 2)

Your Turn: Submit Your Enhanced OOP App

Course Outcome	CO3 — write structured Dart programs using variables, control structures, functions, and object-oriented principles.
Task Type	**Individual Task** — to be completed and submitted by each student on their own
Deliverable	An enhanced OOP console application incorporating inheritance, polymorphism, abstract classes, and mixins.
Ties to	Milestone 2 (Week 12) — this is your most complete OOP artifact before the Dart Logic Practical and written code walkthrough.

Overview

Today you take the class you built in Week 9 and turn it into a small hierarchy — a base class with derived classes that share behavior through inheritance and respond differently through polymorphism. This is an individual task: extend and submit your own program.

Step-by-Step Instructions

Start from your Week 9 class — Open your Week 9 OOP program — the class you built from your ice breaker object.

Create a base class — Turn your Week 9 class into a base class, or design a new one (e.g., Animal, Vehicle, Employee), with at least one field and one method.

Add derived classes — Create at least two derived classes using extends that inherit from your base class.

Override a method — Override at least one method in each derived class so it behaves differently — this demonstrates polymorphism.

Add an abstract class or interface — Include at least one abstract class or interface that defines a method your derived classes must implement.

Add a mixin — Include at least one mixin using with, shared by a class that isn't part of your main inheritance chain.

Submit individually — Submit your own .dart file (or GitHub link, as your instructor specifies) — this is individual work.

Materials & Tools Needed

- Your own Week 9 OOP program (class, constructor, encapsulation)
- DartPad or your local Dart SDK
- Your class notes on inheritance, polymorphism, abstract classes, and mixins from today's lesson

CODE:

DartPad Create Create with Gemini Samples

```dart
abstract class Borrowable {
  void borrowItem();
  void returnItem();
}

mixin Loggable {
  void logTransaction(String action, String title) {
    print('[LOG] ${DateTime.now().toString().split('.')[0]} - Action: $action | Item: "$title"');
  }
}

class Book {
  String title;
  String author;
  double _price;
  bool _isAvailable;

  Book(this.title, this.author, double initialPrice, [bool isAvailable = true])
      : _price = initialPrice,
        _isAvailable = isAvailable;

  double get price => _price;
  set price(double value) => _price = value;

  bool get isAvailable => _isAvailable;
  set isAvailable(bool value) => _isAvailable = value;

  void applyDiscount(double percent) {
    if (percent > 0 && percent <= 100) {
      _price -= _price * (percent / 100);
      print('Applied a $percent% discount to "$title". New price: \$${_price.toStringAsFixed(2)}');
    } else {
      print('Invalid discount percentage.');
    }
  }

  void displayInfo() {
    String status = _isAvailable ? 'Available' : 'Checked Out';
    print('Title: $title | Author: $author | Price: \$${_price.toStringAsFixed(2)} | Status: $status');
  }
}

class PrintedBook extends Book with Loggable implements Borrowable {
  int pageCount;
  String shelfLocation;

  PrintedBook(
    String title,
    String author,
    double price,
    this.pageCount,
    this.shelfLocation, [
    bool isAvailable = true,
  ]) : super(title, author, price, isAvailable);

  @override
  void borrowItem() {
    if (isAvailable) {
      isAvailable = false;
      logTransaction('BORROW_PHYSICAL', title);
      print('Successfully checked out physical copy of "$title" from Shelf $shelfLocation.');
    } else {
      print('Sorry, "$title" is currently unavailable on Shelf $shelfLocation.');
    }
  }

  @override
  void returnItem() {
    isAvailable = true;
    logTransaction('RETURN_PHYSICAL', title);
    print('Returned physical copy of "$title" to Shelf $shelfLocation.');
  }

  @override
  void displayInfo() {
    String status = isAvailable ? 'Available' : 'Checked Out';
    print('[PHYSICAL BOOK] Title: "$title" | Author: $author | Pages: $pageCount | Shelf: $shelfLocation | Price: \$${price.toStringAsFixed(2)} | Status: $status');
  }
}

class EBook extends Book with Loggable implements Borrowable {
  double fileSizeMB;
  String downloadUrl;

  EBook(
    String title,
    String author,
    double price,
    this.fileSizeMB,
    this.downloadUrl, [
    bool isAvailable = true,
  ]) : super(title, author, price, isAvailable);

  @override
  void borrowItem() {
    if (isAvailable) {
      logTransaction('DIGITAL_ACCESS', title);
      print('Access granted for digital copy of "$title". Download link sent: $downloadUrl');
    } else {
      print('Digital access for "$title" is temporarily suspended.');
    }
  }

  @override
  void returnItem() {
    logTransaction('DIGITAL_LICENSE_EXPIRED', title);
    print('Digital access license for "$title" has expired/returned.');
  }

  @override
  void applyDiscount(double percent) {
    double effectivePercent = percent + 5;
    super.applyDiscount(effectivePercent);
  }

  @override
  void displayInfo() {
    print('[E-BOOK] Title: "$title" | Author: $author | File Size: ${fileSizeMB}MB | Price: \$${price.toStringAsFixed(2)} | Link: $downloadUrl');
  }
}

void main() {
  print('=== INITIAL LIBRARY CATALOG ===\n');

  PrintedBook book1 = PrintedBook('1984', 'George Orwell', 15.99, 328, 'A-12');
  EBook book2 = EBook('The Hobbit', 'J.R.R. Tolkien', 22.50, 4.5, 'https://library.org/dl/hobbit');
  PrintedBook book3 = PrintedBook('Clean Code', 'Robert C. Martin', 35.00, 464, 'C-04');

  List<Book> libraryCatalog = [book1, book2, book3];

  for (var book in libraryCatalog) {
    book.displayInfo();
  }

  print('\n=== TESTING OOP FEATURES (POLYMORPHISM, MIXINS, INTERFACES) ===\n');

  book1.applyDiscount(10);
  book2.applyDiscount(10);

  print('');

  book1.borrowItem();
  book2.borrowItem();

  print('');

  book1.returnItem();

  print('\n=== UPDATED CATALOG ===\n');
  for (var book in libraryCatalog) {
    book.displayInfo();
  }
}
```

OUTPUT:

```
=== INITIAL LIBRARY CATALOG ===

[PHYSICAL BOOK] Title: "1984" | Author: George Orwell | Pages: 328 | Shelf: A-12 |
Price: $15.99 | Status: Available
[E-BOOK] Title: "The Hobbit" | Author: J.R.R. Tolkien | File Size: 4.5MB | Price:
$22.50 | Link: https://library.org/dl/hobbit
[PHYSICAL BOOK] Title: "Clean Code" | Author: Robert C. Martin | Pages: 464 | Shelf:
C-04 | Price: $35.00 | Status: Available

=== TESTING OOP FEATURES (POLYMORPHISM, MIXINS, INTERFACES) ===

Applied a 10% discount to "1984". New price: $14.39
Applied a 15% discount to "The Hobbit". New price: $19.13

[LOG] 2026-10-30 10:18:11 - Action: BORROW_PHYSICAL | Item: "1984"
Successfully checked out physical copy of "1984" from Shelf A-12.
[LOG] 2026-10-30 10:18:11 - Action: DIGITAL_ACCESS | Item: "The Hobbit"
Access granted for digital copy of "The Hobbit". Download link sent: https://
library.org/dl/hobbit

[LOG] 2026-10-30 10:18:11 - Action: RETURN_PHYSICAL | Item: "1984"
Returned physical copy of "1984" to Shelf A-12.


=== UPDATED CATALOG ===

[PHYSICAL BOOK] Title: "1984" | Author: George Orwell | Pages: 328 | Shelf: A-12 |
Price: $14.39 | Status: Available
[E-BOOK] Title: "The Hobbit" | Author: J.R.R. Tolkien | File Size: 4.5MB | Price:
$19.13 | Link: https://library.org/dl/hobbit
[PHYSICAL BOOK] Title: "Clean Code" | Author: Robert C. Martin | Pages: 464 | Shelf:
C-04 | Price: $35.00 | Status: Available
```

What to Submit

Submit your .dart file (or GitHub repository link, as your instructor specifies) showing your base class, at least two derived classes, an overridden method, an abstract class or interface, and a mixin, before the end of class or the stated deadline. [Problem-Based Track deliverable] — graded for correctness and hierarchy design.

Scoring Rubric (100 pts)

Criterion	Points	What Earns It
Inheritance	25	At least two derived classes correctly extend a base class.
Polymorphism	25	At least one overridden method makes a derived class behave differently.
Abstract Class/Interface	25	An abstract class or interface defines a contract the derived classes implement.
Mixin	25	At least one mixin is correctly shared with with, outside the main inheritance chain.
TOTAL	**100**	

How This Connects to Milestone 2

Inheritance, polymorphism, abstract classes, and mixins are the exact mechanisms tested on the Milestone 2 practical exam in Week 12, and they're also the mechanisms Flutter itself uses internally — every StatelessWidget you'll write in Week 13 extends a class the same way yours does today.

Reminder
Bring this program to Week 11 — you'll add asynchronous behavior to one of these same classes.